Homework #3

Due: March 16, 2026 at 11:59 PM

Neural Networks and Kernels

Introduction

This homework is about neural networks and kernels.

1. You'll explore how kernel functions implicitly define feature maps and how kernel ridge regression connects to predictions as weighted sums of training labels.
2. You'll derive the backpropagation algorithm for a single-hidden-layer neural network for the binary classification task.
3. You'll discover neural scaling laws empirically and derive optimal compute allocation strategies.

As always, please start early and come to office hours with questions!

Resources and Submission Instructions

You may want to consider the notes from Section 4 (Richer Features and Neural Networks).

Please type your solutions after the corresponding problems using this LaTeX template, and start each problem on a new page.

Please submit the writeup PDF to the Gradescope assignment 'HW3'. Remember to assign pages for each question. **You must include any plots in your writeup PDF.** . Please submit your LaTeXfile and code files to the Gradescope assignment 'HW3 - Supplemental.' The supplemental files will only be checked in special cases, e.g. honor code issues, etc. Your files should be named in the same way as we provide them in the repository, e.g. `hw3.pdf`, etc.

Problem 1 (Kernels and Feature Maps, 30 pts)

In this problem, we explore how kernel functions implicitly define feature maps and how this leads to predictions that are weighted sums of training labels.

Recall from lecture that a kernel function $K(\mathbf{x}, \mathbf{x}')$ computes an inner product in some feature space: $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$, where ϕ is a feature map.

1. **Expanding the polynomial kernel.** Consider the degree-2 polynomial kernel for $\mathbf{x} \in \mathbb{R}^2$:

$$K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^\top \mathbf{x}')^2.$$

   (a) Writing $\mathbf{x} = (x_1, x_2)$ and $\mathbf{x}' = (x_1', x_2')$, expand $(1 + x_1 x_1' + x_2 x_2')^2$ and collect terms. Show that the result can be written as $\phi(\mathbf{x})^\top \phi(\mathbf{x}')$ for some feature map $\phi : \mathbb{R}^2 \to \mathbb{R}^6$. State ϕ explicitly.

   (b) Verify your answer with a concrete example: compute both $K(\mathbf{x}, \mathbf{x}')$ and $\phi(\mathbf{x})^\top \phi(\mathbf{x}')$ for $\mathbf{x} = (1, 0)$ and $\mathbf{x}' = (1, 1)$.

2. **Visualizing the feature map.** We provide a synthetic dataset of concentric circles: an inner ring (class 0) and an outer ring (class 1). See the accompanying notebook for the data generation code.

   (a) Plot the dataset in the original space $\mathbb{R}^2$. Are the data linearly separable? Explain briefly.

   (b) Apply the feature map ϕ from Part 1 to the data. Produce a 3D scatter plot of $(x_1,\ x_2,\ x_1^2 + x_2^2)$ with points colored by class. Are the data linearly separable in feature space? Why does this feature map help for concentric circles?

   (c) Fit a ridge regression model in the feature space:

$$\mathbf{w} = (\mathbf{\Phi}^\top \mathbf{\Phi} + \lambda \mathbf{I})^{-1} \mathbf{\Phi}^\top \mathbf{y},$$

   where $\mathbf{\Phi} \in \mathbb{R}^{N \times 6}$ is the feature matrix with $\phi(\mathbf{x}_n)^\top$ as its nth row and $\mathbf{y} \in \{0, 1\}^N$ are the labels. Use $\lambda = 0.01$. Classify test points by thresholding at 0.5. Use pure `numpy` (no `sklearn`).

3. **From feature space to kernel space.**

   In Part 2c you computed $\mathbf{w}$ and made predictions as $f(\mathbf{x}^*) = \phi(\mathbf{x}^*)^\top \mathbf{w}$, which required explicitly constructing the feature matrix $\mathbf{\Phi}$. We now show that the same predictions can be written as a weighted sum over training points, purely in terms of the kernel function K.

   (a) Let $\mathbf{K} \in \mathbb{R}^{N \times N}$ be the kernel matrix with entries $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$, and define the weight vector $\alpha = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$. Show that the ridge regression prediction at a new point $\mathbf{x}^*$ can be written as:

$$f(\mathbf{x}^*) = \sum_{n=1}^{N} \alpha_n \, K(\mathbf{x}_n, \mathbf{x}^*).$$

   Hint: Define $\mathbf{w} = \mathbf{\Phi}^\top \alpha$ and verify that this $\mathbf{w}$ satisfies the ridge regression normal equations $(\mathbf{\Phi}^\top \mathbf{\Phi} + \lambda \mathbf{I})\mathbf{w} = \mathbf{\Phi}^\top \mathbf{y}$, using $\mathbf{\Phi}\mathbf{\Phi}^\top = \mathbf{K}$.

   (b) Implement kernel ridge regression using the polynomial kernel $K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^\top \mathbf{x}')^2$. Compute $\alpha = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$ on the training data. Then make a scatter plot of the training data where each point is sized proportionally to $|\alpha_n|$.

   (c) Which points have the largest $|\alpha_n|$? Where do they tend to be located? Briefly explain why.

(d) In kernel ridge regression, every training point contributes to the prediction (all $\alpha_n \neq 0$). In a *support vector machine* (SVM), only the training points closest to the decision boundary have $\alpha_n \neq 0$. These points are called *support vectors*.

Looking at your plot from part (b), which training points do you think would be selected as support vectors? Why might it be useful to have most $\alpha_n = 0$?

4. **The RBF kernel.**

The radial basis function (RBF) kernel is defined as:

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right).$$

Unlike the polynomial kernel, whose degree limits it to polynomial decision boundaries, the RBF kernel can approximate any continuous decision boundary. The bandwidth σ controls how quickly $K(\mathbf{x}, \mathbf{x}')$ decays with the distance between $\mathbf{x}$ and $\mathbf{x}'$.

(a) Expand $\|\mathbf{x} - \mathbf{x}'\|^2$ and use the result to write $K(\mathbf{x}, \mathbf{x}')$ as a product of three exponentials: one depending only on $\mathbf{x}$, one only on $\mathbf{x}'$ and one involving $\mathbf{x}^\mathsf{T}\mathbf{x}'$.

(b) The other two factors depend on $\mathbf{x}$ or $\mathbf{x}'$ alone, so they act as scalar weights on each input. The factor involving $\mathbf{x}^\mathsf{T}\mathbf{x}'$ is what couples the two inputs. Expand it as a Taylor series.

(c) The kernel trick lets us work with K without ever computing ϕ explicitly. Why is this essential for the RBF?

(d) Assume the training points are distinct. With $\alpha = (\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{y}$ and $f(\mathbf{x}^*) = \sum_{n=1}^{N} \alpha_n K(\mathbf{x}_n, \mathbf{x}^*)$, show that as $\sigma \to 0$ the model classifies every training point correctly but predicts $f(\mathbf{x}^*) \to 0$ at any point not in the training set.

(e) Show that as $\sigma \to \infty$ the model predicts the same value at every point.

Solution: Your solution here.

Problem 2 (Neural Networks, 20 pts)

In this problem, we will take a closer look at how gradients are calculated for backpropagation with a simple multi-layer perceptron (MLP). The MLP will consist of a first fully connected layer with a sigmoid activation, followed by a one-dimensional, second fully connected layer with a sigmoid activation to get a prediction for a binary classification problem. We use non-linear activation functions as the composition of linear functions is linear. Assume bias has not been merged. Let:

- $\mathbf{W}_1$ be the weights of the first layer, $\mathbf{b}_1$ be the bias of the first layer.
- $\mathbf{W}_2$ be the weights of the second layer, $\mathbf{b}_2$ be the bias of the second layer.

The described architecture can be written mathematically as:

$$\hat{y} = \sigma(\mathbf{W}_2 \left[\sigma\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right)\right] + \mathbf{b}_2)$$

where $\hat{y}$ is a scalar output of the net when passing in the single datapoint $\mathbf{x}$ (represented as a column vector), the additions are element wise additions, and the sigmoid is an element wise sigmoid.

1. Let:

   - N be the number of datapoints we have
   - M be the dimensionality of the data
   - H be the size of the hidden dimension of the first layer. Here, hidden dimension is used to describe the dimension of the resulting value after going through the layer. Based on the problem description, the hidden dimension of the second layer should be 1.

   Write out the dimensionality of each of the parameters, and of the intermediate variables:

$$\mathbf{a}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{z}_1 = \sigma(\mathbf{a}_1)$$
$$a_2 = \mathbf{W}_2\mathbf{z}_1 + \mathbf{b}_2, \qquad \hat{y} = z_2 = \sigma(a_2)$$

   and make sure they work with the mathematical operations described above. Examining shapes is one of the key ways to debug your code, and can be done using .shape after any numpy array.

2. We will derive the gradients for each of the parameters, which can then be used along with gradient descent to find weights that improve our model's performance. For this question, assume there is only one datapoint $\mathbf{x}$, and that our loss is $L = -(y\log(\hat{y}) + (1-y)\log(1-\hat{y}))$. For all questions, the chain rule will be useful.

   (a) Find $\frac{\partial L}{\partial b_2}$.

   (b) Find $\frac{\partial L}{\partial W_2^h}$, where W_2^h represents the hth element of $\mathbf{W}_2$.

   (c) Find $\frac{\partial L}{\partial b_1^h}$, where b_1^h represents the hth element of $\mathbf{b}_1$. (**Hint**: Note that only the hth element of $\mathbf{a}_1$ and $\mathbf{z}_1$ depend on b_1^h - this should help you with how to use the chain rule.)

   (d) Find $\frac{\partial L}{\partial W_1^{h,j}}$, where $W_1^{h,j}$ represents the element in row h, column j in $\mathbf{W}_1$.

Problem 2 (cont.)

3. We now explore an example of forward-mode auto-differentiation. Consider the following equation:

$$f(x_1, x_2) = \ln(\sin(x_1)) + x_1 \exp\{x_2\}$$

This equation can be split up using intermediate variables $v_1, \ldots, v_7$ as follows:

$$\begin{aligned} v_1 &= x_1 \\ v_2 &= \sin(v_1) \\ v_3 &= \ln(v_2) \\ v_4 &= x_2 \\ v_5 &= \exp\{v_4\} \\ v_6 &= v_1 v_5 \\ v_7 &= v_3 + v_6 \\ f(x_1, x_2) &= v_7 \end{aligned}$$

Splitting up the equation like this is very similar to what an auto-differentiation library would do. From these equations we can construct a *computational graph* where each node of the graph corresponds to an input, an intermediate variable, or the output.

(a) Let $x_1 = \frac{\pi}{6}$ and $x_2 = 1$. Calculate the values of all the intermediate variables $v_1, \ldots v_7$ and $f(x_1, x_2)$.

(b) Calculate the derivative of all of the intermediate variables $v_1, \ldots, v_7$ and f with respect to x_1 evaluated at $x_1 = \frac{\pi}{6}$ and $x_2 = 1$. Remember to write out the equations before evaluating, e.g.,

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(x)}{\partial g(x)} \frac{\partial g(x)}{\partial x}.$$

Solution: Your solution here.

Problem 3 (Neural Scaling Laws, 50 pts)

In this problem, we look pr an observation that is extremely relevant and widely utilized in industry and research.

Large-scale experiments have revealed a striking observation: the test loss of neural networks follows **power laws** in model size and dataset size. In this problem, you will discover these laws yourself on a smaller scale, use them to make predictions, and derive the optimal way to allocate a fixed compute budget. Empirically, for many architectures and datasets, test loss L follows:

$$L(N, D) \approx \frac{a}{N^\alpha} + \frac{b}{D^\beta} + L_\infty \tag{1}$$

where N is the number of trainable parameters, D is the number of training samples, and L_∞ is the irreducible error (the best loss any model could achieve on this task). On a log-log plot, each term appears as a straight line. There is no first-principles derivation of *why* this particular functional form holds, it is an empirical observation. However, the form makes intuitive sense through the lens of bias-variance:

- a/N^α: **approximation error**. A small model cannot represent the true function well. As we add parameters, this error decreases — but with diminishing returns (power law, not exponential).
- b/D^β: **estimation error**. Even a large model cannot learn well from too little data. More data reduces this error, again with diminishing returns.
- L_∞: **irreducible error**. Some loss remains no matter how large the model or dataset — due to label ambiguity, insufficient input information (e.g., low-resolution images), or inherent noise.

Setup: Architecture

You will use a depth-scaled residual CNN (ResNet) with a fixed base channel width $C = 64$ and a variable number of residual blocks K:

Stem: $\texttt{Conv2d}(1, C, 3, \text{padding} = 1, \text{bias} = \text{False}) \to \texttt{BatchNorm2d}(C) \to \texttt{ReLU} \to \texttt{MaxPool2d}(2)$

$K\times$ **ResBlock:** each block consists of

$$\texttt{Conv2d}(C, C, 3, \text{padding} = 1, \text{bias} = \text{False}) \to \texttt{BatchNorm2d}(C) \to \texttt{ReLU}$$
$$\to \texttt{Conv2d}(C, C, 3, \text{padding} = 1, \text{bias} = \text{False}) \to \texttt{BatchNorm2d}(C) + \text{skip} \to \texttt{ReLU}$$

Head: $\texttt{AdaptiveAvgPool2d}(1) \to \texttt{Flatten} \to \texttt{Linear}(C, 10)$

Fashion-MNIST images are 28×28 with 1 channel. After the stem's max-pool the spatial size is 14×14; the residual blocks preserve this size; the adaptive average pool reduces it to 1×1. You are free to use `torch.nn` and `torchvision` for this problem.

(a) **[22 pts]** Train ResNets on Fashion-MNIST at varying scales.

(i) **[10 pts]** Write out the total parameter count N as a function of K (number of residual blocks) for the architecture above. Be careful: each `Conv2d` with `bias=False` has $\text{kernel}^2 \times C_{\text{in}} \times C_{\text{out}}$ parameters, each $\texttt{BatchNorm2d}(C)$ has $2C$ parameters (scale and shift), and the linear layer has in_features $\times$ out_features + out_features. Write the training code for the ResNet. The values for the different K values are provided for your convenience.

(ii) **[6 pts]** Fix $K = 12$ and train on random subsets of size $D \in \{1000, 2000, 5000, 10000, 20000, 30000, 50000\}$. Record the final test loss for each.

(iii) **[6 pts]** Make two log-log plots:

- $\log(L - \hat{L}_\infty)$ vs. $\log N$ (at fixed D)
- $\log(L - \hat{L}_\infty)$ vs. $\log D$ (at fixed N)

where $\hat{L}_\infty$ is estimated as the loss of the best configuration *within each sweep* (i.e., the $K = 18$ run for the model-size plot, and the $D = 50000$ run for the data-size plot). This anchor point is excluded from the fit in each case. Fit a line to each plot. Report your estimated exponents $\hat{\alpha}$ and $\hat{\beta}$, the R^2 values, as well as your plots, down below. Discuss briefly: why do we need to subtract $\hat{L}_\infty$ before taking the log? What would the plot look like if we skipped this step? Feel free to modify the graphing code to see what it would look like, though that is not necessary.

(b) **[10 pts]** Suppose training costs C_{compute} FLOPs, and assume the approximation $C_{\text{compute}} \approx 6ND$ (forward + backward pass ≈ 6 multiply-adds per parameter per sample).

(i) **[5 pts]** For a fixed compute budget C_{compute}, we want to choose N and D to minimize the loss in Eq. (**??**). Using the constraint $D = C_{\text{compute}}/(6N)$, substitute into the loss and minimize over N. Show that the compute-optimal parameter count satisfies:

$$N^*(C_{\text{compute}}) \propto C_{\text{compute}}^{\beta/(\alpha+\beta)}$$

and the compute-optimal dataset size satisfies:

$$D^*(C_{\text{compute}}) \propto C_{\text{compute}}^{\alpha/(\alpha+\beta)}.$$

(ii) **[3 pts]** Plug in your empirically estimated $\hat{\alpha}$ and $\hat{\beta}$ from part (a). If you double your compute budget, by what factor should you increase N? By what factor should you increase D?

(iii) **[2 pts]** The "Chinchilla" result (Hoffmann et al., 2022) found $\alpha \approx \beta$ for large language models, implying N and D should be scaled equally. If instead $\alpha \gg \beta$ (model size matters much more than data), what would the optimal strategy look like? What about $\alpha \ll \beta$?

(c) **[8 pts]** This is the key test of whether scaling laws are actually useful: can you predict the performance of a model you haven't trained yet?

(i) **[4 pts]** Using *only* your fitted scaling law from part (a), predict the test loss at $D = 60000$. Then actually train that model and report the true test loss. How close was your prediction? Discuss potential reasons for any discrepancy.
Hint: What problems may arise from using $D = 50000$ as your irreducible loss, and what value does it get?

(ii) **[4 pts]** The $K = 18$ model trained on the full dataset achieved a test cross-entropy of 0.2195. Suppose you are limited to the $K = 12$ architecture. According to your fitted data scaling law, how much training data would $K = 12$ need to match $K = 18$'s performance? Is this achievable? What does this tell you about the relationship between model capacity and data?

Solution: Your solution here.

Name:
Collaborators and Resources: